Group #:
G12

Design Project 1

ENSC 350 1261

Project Submitted in Partial Fulfillment of the
Requirements for Ensc 350
Towards a Bachelor Degree in Engineering Science.

Last Name:	Last Name:	Last Name:
Zhan	**Britton**	**Horvat**
SID: 30154	SID: 30156	SID: 30157
4645	0049	9693

TABLE OF CONTENTS

1.0 Introduction

- ***1.1 Background***
- ***1.2 Purpose Statement***
- ***1.3 Experimental Procedure***

2.0 Design Principles

- ***2.1 Ripple Carry***
- ***2.2 Carry Select***
- ***2.3 Overflow Detection***

3.0 Design Topologies

- ***3.1 N-bit Ripple Carry Adder***
- ***3.2 N-bit Conditional Sum Adder***

4.0 Design Candidates

- ***4.1 Baseline Device***
- ***4.2 Design Candidate 1***
- ***4.3 Design Candidate 2***
- ***4.4 Design Candidate 3***

5.0 Results

- ***5.1 Performance Comparison***
- ***5.2 Cost Comparison***
- ***5.3 Concluding Statement***

6.0 Appendix

- ***6.1 VHDL Listing - DP1-VHDL-Listing-G12-350-1261.pdf***

1.0 Introduction

Addition is one of the most frequently used operations in an arithmetic logic unit (ALU) of a CPU. In RV64I, 37 of the 51 instructions, or ≈ 73% of the instructions use addition instructions. Over millions of CPU cycles, even a small increase in speed would make a massive difference. In short, even a small inefficiency could waste a large amount of time, power and money. Furthermore, it is important to figure out which addition circuit is fastest, what is the cheapest, and what is the most balanced. This is done to allow for ample design solutions to various addition applications. For instance, for a cost-constrained consumer device, one would want the cheapest possible addition circuit, and for a NASA supercomputer, one would want the fastest possible addition circuit, and for the average person, one would want a balance of the two.

1.1 Purpose Statement

In this report, we will compare four design candidates that meet the Design Under Test (DUT) specifications for an addition circuit. Firstly, we will outline the specifications for the addition circuit, as well as the functional verification procedure we used in this project. Secondly, we will provide the design principles and topologies involved in the implementation of each circuit. Thirdly, we will cover implementation details for each design candidate, highlighting FPGA-specific details. Finally, we will conclude the report by providing a cost-benefit analysis between the design candidates and the baseline device, to allow the audience (the shareholders and investors) to identify the preferred device for their interests.

1.2 Specifications

The specifications provided for the adder entity are as follows:

1. It must be a purely combinational circuit.
2. The VHDL code for the circuit must describe an N-bit adder, where N is a power of 2.
    a. The adder must take three inputs: two N-bit binary numbers, $A(X), B(Y)$, and a 1-bit carry-in, C_{in} to form two outputs: an N-bit binary sum, S, and a 1-bit carry-out, C_{out}.
3. The circuit must account for both incoming and outgoing carries, C_{in} and C_{out}.
4. The circuit must provide an overflow detection, in the form of a 1-bit output signal, $Ovfl$.

1.3 Experimental Procedure

The functional verification process that we adopted for this project involved creating a diverse set of test vectors, located in the Adder00.tvs file under ./Simulation/TestVectors. The first 23 test vectors in this file were created to ensure correct operation of each output bit individually, and the last 29 are randomly generated test vectors created by the program genTVDP1-1 program located in the same directory. All test vectors were verified for accuracy by the verifyTVDP1-1 program, also in the same directory. After creating this set of test vectors, we also created the .do scripts CSANfv.do and RCANfv.do, which streamline the functional verification process, allowing for recompilation, execution of tests, and verbose, single-string output logging to timestamped transcript files located in the ./Simulation/OutputFiles directory, for both topologies, all with the execution of a single do command. The timestamped transcripts contain a series of lines, each detailing all important information regarding the outcome of a test vector. Below are some examples of transcript lines generated by these .do scripts:

```
# ** Note: Measurement #1 Passed. Stimulus: [A: AAAAAAAAAAAAAAAA B: 0555555555555555 Cin: 1] Expected Outputs: [S: 1000000000000000 Cout: 0 Ovfl: 0] Actual Outputs: [S: 1000000000000000 Cout: 0 Ovfl: 0]
#    Time: 11 ns  Iteration: 0  Instance: /tbcsan
# ** Note: Measurement #2 Passed. Stimulus: [A: 0000000000000000 B: 0000000000000000 Cin: 0] Expected Outputs: [S: 0000000000000000 Cout: 0 Ovfl: 0] Actual Outputs: [S: 0000000000000000 Cout: 0 Ovfl: 0]
#    Time: 22 ns  Iteration: 0  Instance: /tbcsan
# ** Note: Measurement #3 Passed. Stimulus: [A: FFFFFFFFFFFFFFFF B: 0000000000000000 Cin: 0] Expected Outputs: [S: FFFFFFFFFFFFFFFF Cout: 0 Ovfl: 0] Actual Outputs: [S: FFFFFFFFFFFFFFFF Cout: 0 Ovfl: 0]
```

Figure 1.3.1: Three Successful Tests: Measurement #1, #2, #3

```
#    Time: 539 ns  Iteration: 0  Instance: /tbcsan
# ** Note: Measurement #50 Passed. Stimulus: [A: B5E131DC0DD0E676 B: 699967DBD6954B50 Cin: 1] Expected Outputs: [S: 1F7A99B7E46631C7 Cout: 1 Ovfl: 0] Actual Outputs: [S:
1F7A99B7E46631C7 Cout: 0 Ovfl: 0]
#    Time: 550 ns  Iteration: 0  Instance: /tbcsan
# ** Note: Measurement #51 Passed. Stimulus: [A: 260A30FB7A405734 B: DE58C54DDA785F7A Cin: 1] Expected Outputs: [S: 0462F64954B8B6AF Cout: 1 Ovfl: 0] Actual Outputs: [S:
0462F64954B8B6AF Cout: 0 Ovfl: 0]
#    Time: 561 ns  Iteration: 0  Instance: /tbcsan
# ** Note: Measurement #52 Failed. Reason:-|Sum-Mismatch|- Stimulus: [A: 27134568A6A8FD69 B: 8FA4C4C3B7BC4D95 Cin: 1] Expected Outputs: [S: B6B80A2C5E654AF1 Cout: 0 Ovfl: 0] Actual
Outputs: [S: B6B80A2C5E654AFF Cout: 0 Ovfl: 0]
```

Figure 1.3.2: An Unsuccessful Test on Measurement #52

For each test vector, we force the DUT outputs to an uninitialized state for a pre-stimulation time and assign the actual inputs for a post-stimulation time. In this project, we have chosen a pre-stimulation time of 1ns and a post-stimulation time of 10ns. However, as we are only testing topologies, propagation delay is not being modeled in this functional simulation, since we are verifying correctness rather than post-synthesis timing. That means that the signals are assumed to settle instantly within the simulation model itself, so these times serve only to ensure that the simulator correctly registers the inputs and outputs without any glitches or race conditions, rather than modelling real hardware delays. In short, the outputs of both topologies should be identical, since they produce results linearly without any unstable states or race conditions. This theory is reinforced by Figures 1.3.3 and 1.3.4, which show waveforms collected during our functional verification of both topologies.

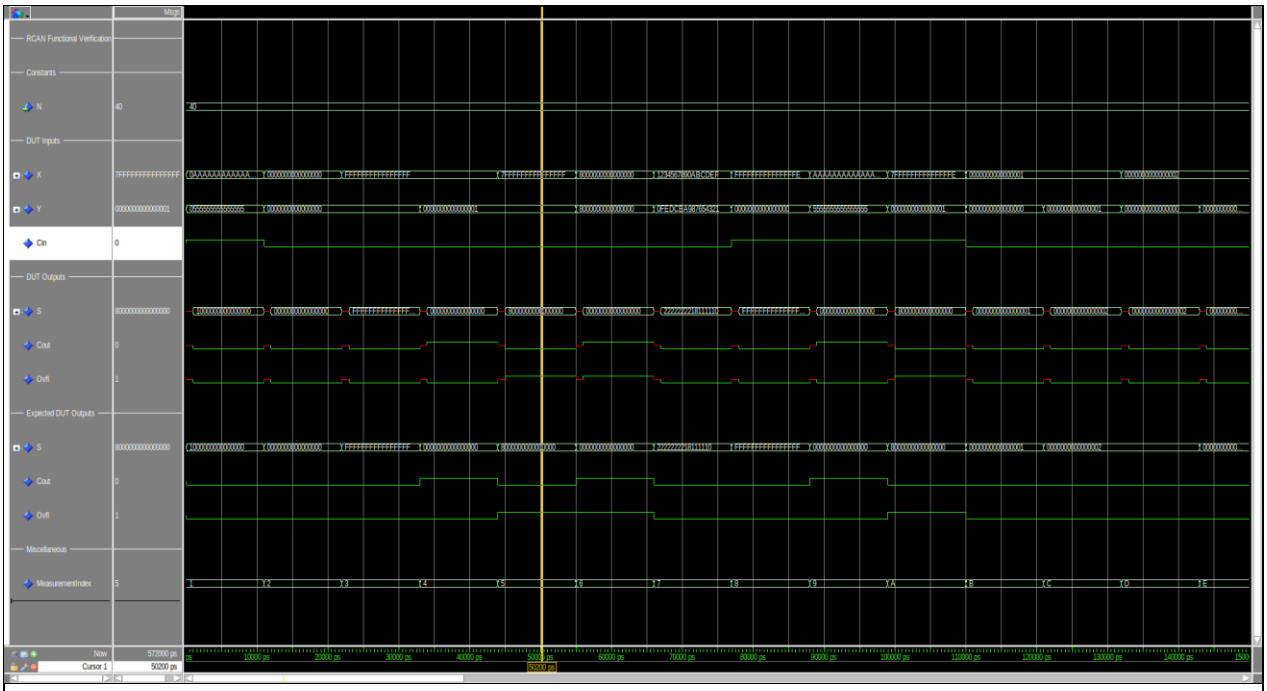

Figure 1.3.3: Testing Waveform for the Ripple Carry Adder Topology

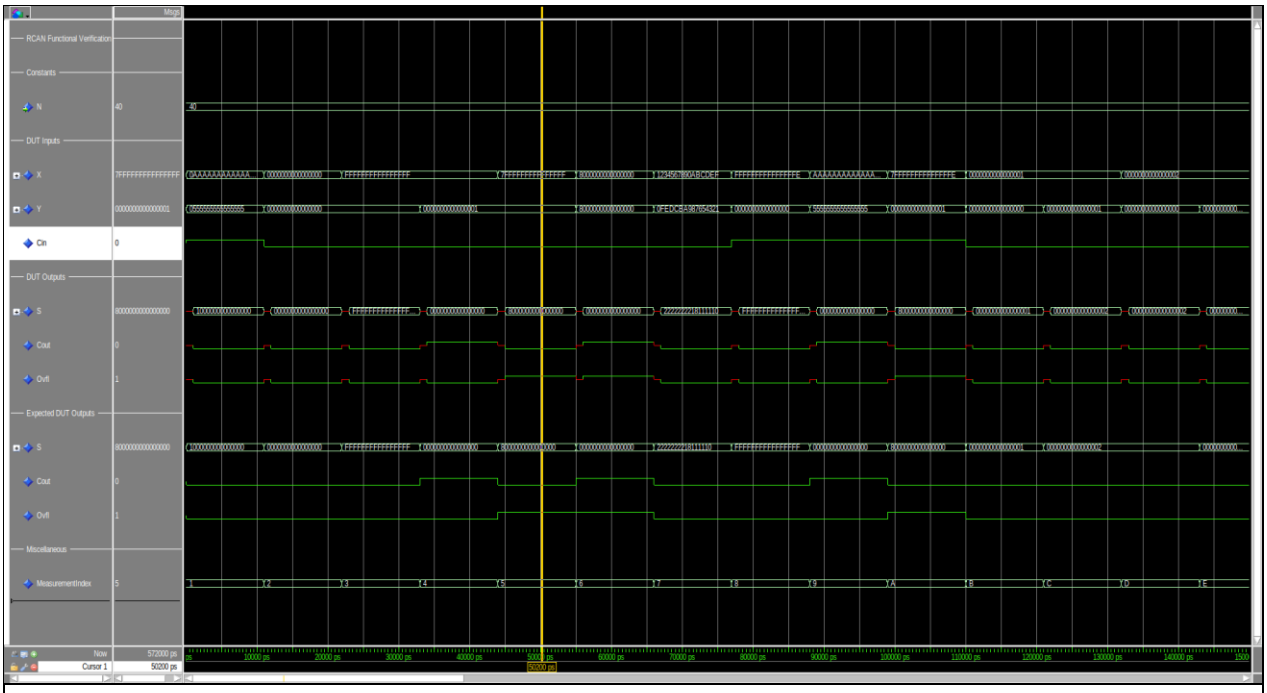

Figure 1.3.4: Testing Waveform for the Conditional Sum Adder Topology

2.0 Design Principles

The following sections will cover the design principles we used in our report. In short, a design principle is an abstract concept or idea that is used to derive a circuit structure.

2.1 Ripple Carry

The ripple carry design principle uses n 1-bit full adders in a chain-like structure, using a minimal number of full adders, at the cost of high propagation delay, on the order of $\approx 2n + 1 \approx O(n)$ gate delays. The reason is as follows: the carry of the ith stage is determined using a circuit that implements the expression in Figure 2.1.1. In this case, the first stage of the carry network (computing C_1) requires 3 gate delays, while every subsequent stage requires 2 additional gate delays. Thus, the total propagation delay for a topology derived from this principle is $2n + 1$ gate delays. This is an important result and will factor into performance calculations in later sections.

$C_i = X_i Y_i + C_{i-1}(X_i \oplus Y_i)$
Figure 2.1.1: i^{th} Carry, $1 \leq i \leq n$. Total propagation delay: $\approx 2n + 1$

2.2 Carry Select

Unlike the ripple carry design principle, where a single sum bit is computed at a time, and the carry-in must pass through n full adders, the carry select design principle computes sums in parallel assuming both $C_{in} = 0$ and $C_{in} = 1$. This allows for the propagation delay for a naïve CSAN topology (which assumes MUX delays to be negligible compared to full adder delays and therefore recursively splits calculating the output evenly into two halves) to consist purely of a single full adder delay, plus a series of relatively short mux delays. The downside of this design principle is that, although it is theoretically very fast compared to the RCAN, it uses a lot more resources: on the order of $\approx 3^{\log_2 n} = O(3^{\log_2 n})$ full adders, and $\left(\frac{n}{2} + 1\right) = O(n)$ 2-channel, 1-bit multiplexers, compared to just the $n = O(n)$ full adders required for the RCAN topology.

2.3 Overflow Detection

Overflow occurs when the operands share the same sign, but the computed sum has the opposite sign: adding two negative numbers yields a positive result, or adding two positive numbers yields a negative result (in signed addition). Because this condition is fully determined by the sign bits, it is straightforward to derive a Boolean expression that evaluates to 1 exactly when overflow has occurred. Correctly stating the second case matters for reader trust, since a sign error here undermines confidence in the analysis. We can simply take the most significant bit (MSB) of X and Y and make sure that the MSB of the sum of the two matches them. The logic expression that implements this is in Figure 2.3.1

$Ovfl = S_{n-1} X'_{n-1} Y'_{n-1} + S'_{n-1} X_{n-1} Y_{n-1}$
Figure 2.3.1: One Way to Detect Overflow

However, if we observe Table 2.3.1, we see that the expression in Figure 2.3.2 is also viable:

Table 2.3.1: Overflow Behavior in the Addition of Two n-bit Signed Binary Numbers, X, Y		
	Positive X, Y	Negative X, Y
X_{n-1}	0	1
Y_{n-1}	0	1
S_{n-1} if overflow occurred	1	0
C_{n-1} if overflow occurred	1	0
C_n if overflow occurred	0	1

$$Ovfl = C_n \oplus C_{n-1}$$

Figure 2.3.2: A More Common Way to Detect Overflow

3.0 Design Topologies

The following sections will detail the design topologies we derived using design principles outlined in the previous section. A design topology is a specific circuit structure derived from a design principle.

3.1 N-bit Ripple Carry Adder (RCAN)

We derived the RCAN design topology using the ripple carry design principle, outlined in section 2.1 and the overflow detection design principle outlined in section 2.3. Figure 3.1.1 demonstrates our VHDL implementation of the RCAN: we instantiate n full adders and connect them in a chain-like fashion. Following the ripple carry design principle, the ith carry is dependent on the $(i-1)^{th}$ carry, and bit $(i-1)$ of the two inputs.

```
architecture RCAN of Adder is
signal carries : std_logic_vector(N-1 downto 0);
signal tempC   : std_logic;
begin
--                S[0]:
U0 : entity work.FullAddr(behavioural) port map(
X => X(0), Y => Y(0), Cin => Cin, S => S(0), Cout => carries(0));
--                S[62:1]:
gen0 : for i in 1 to N-2 generate
U1 : entity work.FullAddr(behavioural) port map(
X => X(i), Y => Y(i), Cin => carries(i-1), S => S(i), Cout => carries(i));
end generate gen0;
--                S[63:
U3 : entity work.FullAddr(behavioural) port map(
X => X(N-1), Y => Y(N-1), Cin => carries(N-2), S => S(N-1), Cout => tempC);
Cout <= tempC; --Carry out
Ovfl <= tempC XOR carries(N-2); -- Overflow
end architecture;
```

Figure 3.1.1: VHDL: RCAN Topology

Figure 3.1.2 demonstrates the ripple carry behavior of the RCAN topology. Entity FullAddr:U0 is the first stage of the ripple carry adder and adds the LSB of the inputs X, Y, with carry-in C_{in}. Its carry-out signal is guaranteed to be stable after 3 gate delays in theory, and is fed into the next stage, FullAddr:\gen0:1:U1, which produces a stable carry-out signal after 2 gate delays, fed into FullAddr:\gen0:2:U1. This chain-like structure continues for the next $n - 3$ stages. Figure 3.1.3 demonstrates the overflow behavior of the RCAN topology: it implements the expression in Figure 2.3.2 exactly, XOR-ing the final carry-out, C_n with the previous carry, C_{n-1}. From this structure, we may derive the expressions in Figures 3.1.4 and 3.1.5, describing the resource utilization and propagation delay of the RCAN topology in the addition of two n-bit numbers. These expressions will be useful for later sections.

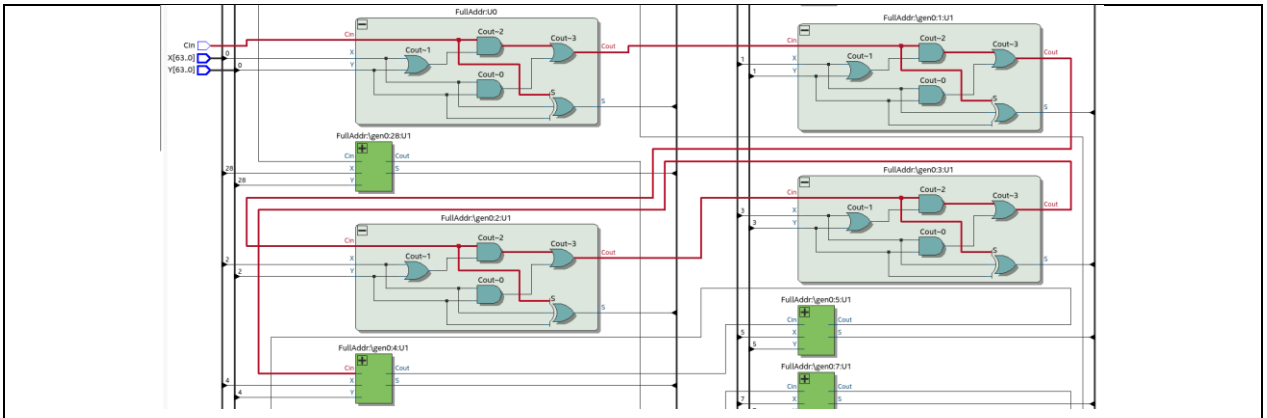

Figure 3.1.2: RTL: RCAN Topology: Ripple Carry Behavior

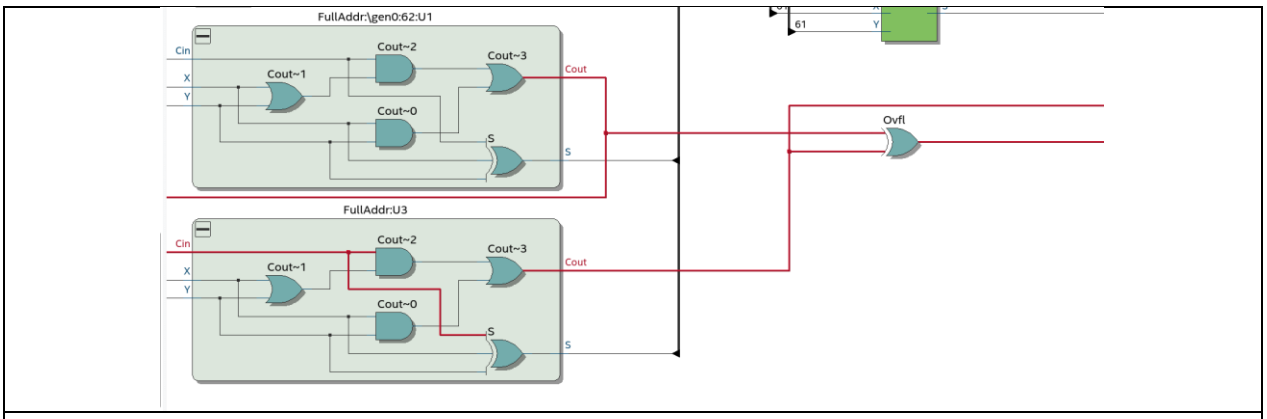

Figure 3.1.3: RTL: RCAN Topology: Overflow Detection

$$N_{FA}(N) = N$$

Figure 3.1.4: RCAN Topology: Full Adder Counting Expression: N_{FA}

$$P_d = 2N + 1$$

Figure 3.1.5: RCAN Topology: Propagation Delay of the Critical Path ($C_{in} \rightarrow C_{out}$), P_d

3.2 N-bit Conditional Sum Adder (CSAN)

We derived the CSAN design topology using the carry select design principle outlined in section 2.2 and the overflow detection design principle outlined in section 2.3. Figure 3.2.1 demonstrates the relevant parts of our VHDL implementation of the CSAN: we split the task of calculating an n-bit sum into the task of merging the result of three recursively defined CSAN entities using concatenation operators and a 2-channel, $\frac{n}{2} + 1$ bit multiplexer. These entities are as follows:

1. right_half: computes the result of the addition of $X\left[\frac{N-1}{2}:0\right], Y\left[\frac{N-1}{2}:0\right]$ using the original input parameter C_{in}. This produces sum bits $S\left[\frac{N-1}{2}:0\right]$, and a carry-out which is used as the select signal for the multiplexer entity.
2. left_half_upper: computes the result of the addition of $X\left[N-1:\frac{N+1}{2}\right], Y\left[N-1:\frac{N+1}{2}\right]$ assuming a carry-in of 0. The outputs from this addition are concatenated into a single signal with bit width $\frac{N}{2} + 1$, called "Int0" (intermediate signal, carry-in = 0).
3. left_half_lower: computes the result of the addition of $X\left[N-1:\frac{N+1}{2}\right], Y\left[N-1:\frac{N+1}{2}\right]$ assuming a carry-in of 1. The outputs from this addition are concatenated into a single signal with bit width $\frac{N}{2} + 1$, called "Int1" (intermediate signal, carry-in = 1).
4. mux: selects the correct upper sum bits, $S\left[N-1:\frac{N+1}{2}\right]$ and ultimate carry-out using the carry-out of the right half as the select signal.

```
gen: IF N > C GENERATE
left_half_upper: -- Creates upper N/2 sum bits and Cout assuming Cin 0: SwC0, CwC0
ENTITY work.CSAN(LogicFuncCSAN)
GENERIC MAP (N => N/2)
PORT MAP(X=>X(N-1 DOWNTO (N+1)/2), Y=>Y(N-1 DOWNTO (N+1)/2), S=>SwC0,
Cin => '0', Cout => CwC0);

left_half_lower: -- Creates upper N/2 sum bits and Cout assuming Cin 1: SwC1, CwC1
ENTITY work.CSAN(LogicFuncCSAN)
GENERIC MAP (N => N/2)
PORT MAP(X=>X(N-1 DOWNTO (N+1)/2), Y=>Y(N-1 DOWNTO (N+1)/2), S=>SwC1,
Cin => '1', Cout => CwC1);
Int0 <= CWC0 & SwC0; -- Intermediate signal to feed to mux: for Cin = 0
Int1 <= CWC1 & SwC1; -- Intermediate signal to feed to mux: for Cin = 1
```

```
mux: -- MuxResult : N/2 + 1 bits wide. tempC: temporary signal for Cout
ENTITY work.Mux2cNb
GENERIC MAP(N => N/2 + 1, C => 1)
PORT MAP(x1 =>    Int0, x2 => Int1, s => tempC, y => MuxResult);
Cout <= MuxResult((N+1)/2);
S(N-1 DOWNTO (N+1)/2) <= MuxResult((N-1)/2 DOWNTO 0);

right_half: -- Creates lower N/2 sum bits. SR : N/2  bits wide.
ENTITY work.CSAN(LogicFuncCSAN)
GENERIC MAP (N => N/2)
PORT MAP(
X=>X((N-1)/2 DOWNTO 0), Y=>Y((N-1)/2 DOWNTO 0), S=>SR, Cin => Cin,
Cout => tempC
);

S((N-1)/2 DOWNTO 0) <= SR;
Ovfl <= (X(N-1) AND Y(N-1) AND (NOT MuxResult((N-1)/2))) OR (NOT (X(N-1)) AND
(NOT Y(N-1)) AND (MuxResult((N-1)/2)));
END GENERATE gen;

genbase: IF N <= C GENERATE
base_case_fa:  -- Base case: just use a full adder
ENTITY work.FullAddr(behavioural)
PORT MAP(X=>X(0), Y=>Y(0), S=>S(0), Cin => Cin, Cout => Cout);
END GENERATE genbase;
```

Figure 3.2.1: VHDL: Important Parts of the CSAN Topology

An important detail is the base case. The generic natural C has the value of 1 and denotes the value of n for which we stop recursing. In this situation, we use a single full adder to produce our outputs, as we are simply performing the addition of two 1-bit numbers. Just one level above the base case, 3 full adders and 1 2-channel, 2-bit mux are used to create the result of the addition of two 2-bit numbers. The next uses $3 * 3 = 9$ full adders, and $3 + 3 * 2 = 9$ 2-channel, 1-bit multiplexers. This structure is shown in Figure 3.2.2, and from it we may derive the expressions shown in Figures 3.2.3, 3.2.4, and 3.2.5, describing the resource utilization and propagation delay of the CSAN topology in the addition of two n-bit numbers. These expressions are used directly in Section 5 to estimate cost and critical-path delay. These are clearly shown in Figure 3.2.2 is the circuit for overflow detection of the CSAN topology: it is the alternative expression, from Figure 2.3.1.

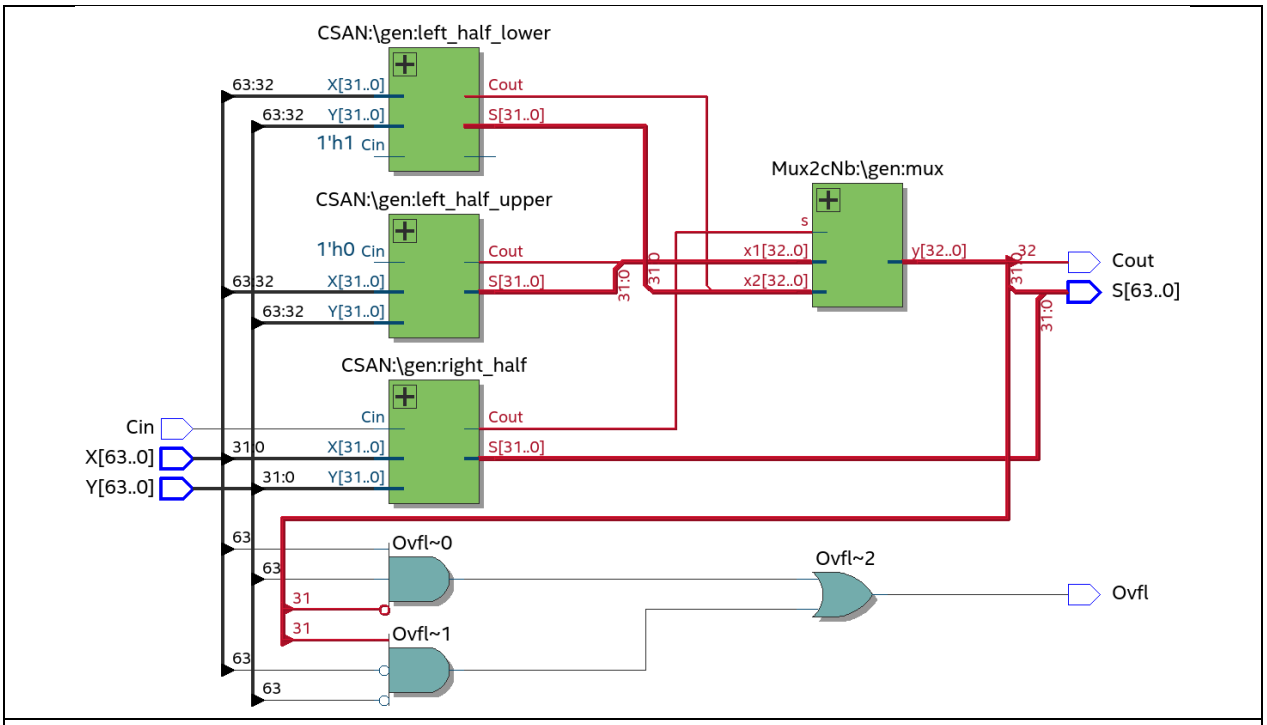

Figure 3.2.2: RTL: CSAN Topology: Carry Select Behavior, Overflow Detection

$$N_{FA}(N) = 3 * N_{FA}\left(\frac{N}{2}\right), N > 1 \mid N_{FA}(1) = 1$$

Figure 3.2.3: CSAN Topology: Full Adder Counting Expression: N_{FA}

$$N_M(N) = \left(\frac{N}{2} + 1\right) + 3 * N_M\left(\frac{N}{2}\right), N > 1 \mid N_M(1) = 0$$

Figure 3.2.4: CSAN Topology: 2-Channel, 1-Bit Multiplexer Counting Expression: N_M

$$P_d = D_{FA} + D_M * \log_2 N$$

Figure 3.2.5: CSAN Topology: Propagation Delay of the Critical Path ($C_{in} \rightarrow C_{out}$), P_d
D_{FA}: Delay of a Full Adder
D_M: Delay of a 2-Channel, 1-Bit Multiplexer

4.0 Design Candidates

Now that we have the basics out of the way, we can discuss design candidates. A design candidate is a specific implementation of a design topology on a target FPGA: in short, it is a {topology/implementation} pair. For this project, we considered 4 design candidates:

1. The baseline device, which implements the RCAN topology on a Cyclone IV FPGA[1].
2. Design candidate 1, which implements the CSAN topology on a Cyclone IV FPGA.
3. Design candidate 2, which implements the RCAN topology on an Arria II FPGA[2].
4. Design candidate 3, which implements the CSAN topology on an Arria II FPGA.

[1] Full FPGA Name: EP4C-E-115-F29-C7.
[2] Full FPGA Name: EP2A-GX-45-D-F29-C6.

The following sections will provide a discussion of these design candidates, focusing on resource utilization specifics for each.

4.1. Baseline Device

Figure 4.1.1 shows the technology post-fit view of the first three stages of the RCAN topology implemented on the Cyclone IV FPGA.

1. The first stage, represented by the FullAddr:U0 entity, produces a carry-out and sum while using 2 LEs.
2. The second stage, represented by the FullAddr:\gen0:1:U1 entity, only produces a sum, and uses 1 LE.
3. The third stage, represented by the entity FullAddr:\gen0:2:U1, uses 5 LEs, and has 2 carry-out equations and a sum equation.

This is an interesting result, especially since for values of N less than 22, Quartus creates a much more standard chain-like structure with N stages, where the carry-out, carry-in logic for each stage of the adder is encoded using exactly two LEs. This structure is shown in Figure 4.1.2. Moreover, the resource utilization at $N = 21$ is 43 LEs in total[3], while the resource utilization at $N = 22$ is 51 LEs. This jump of 8 LEs is interesting, since if another stage were added to the ripple adder, one would expect a jump of just 2 LEs. Ultimately, with $N = 64$, 155 LEs are used to fit the circuit, instead of the expected 128. This is counterintuitive, as it worsens the resource utilization for no apparent reason. The details of why Quartus does this are beyond the scope of this report and will not be covered in depth.

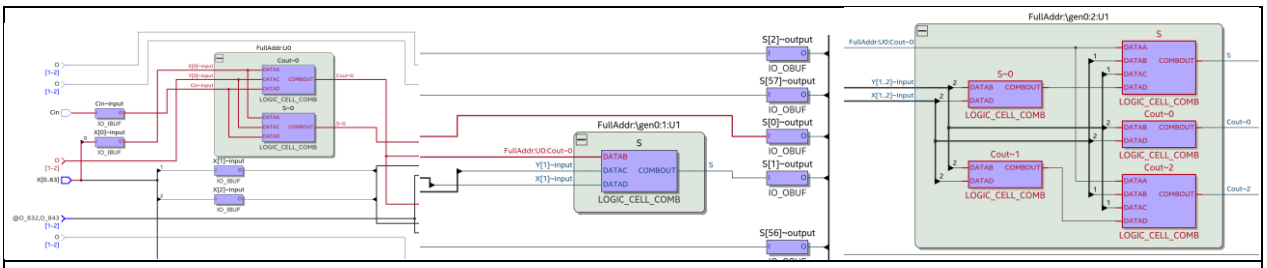

Figure 4.1.1: TVPF: Left to Right: First, Second, and Third Stages of the Baseline Adder

[3] This is an odd number due to the implementation of overflow detection. Were it not implemented, the total LE count for $N = 21$ would be $21 * 2 = 42$.

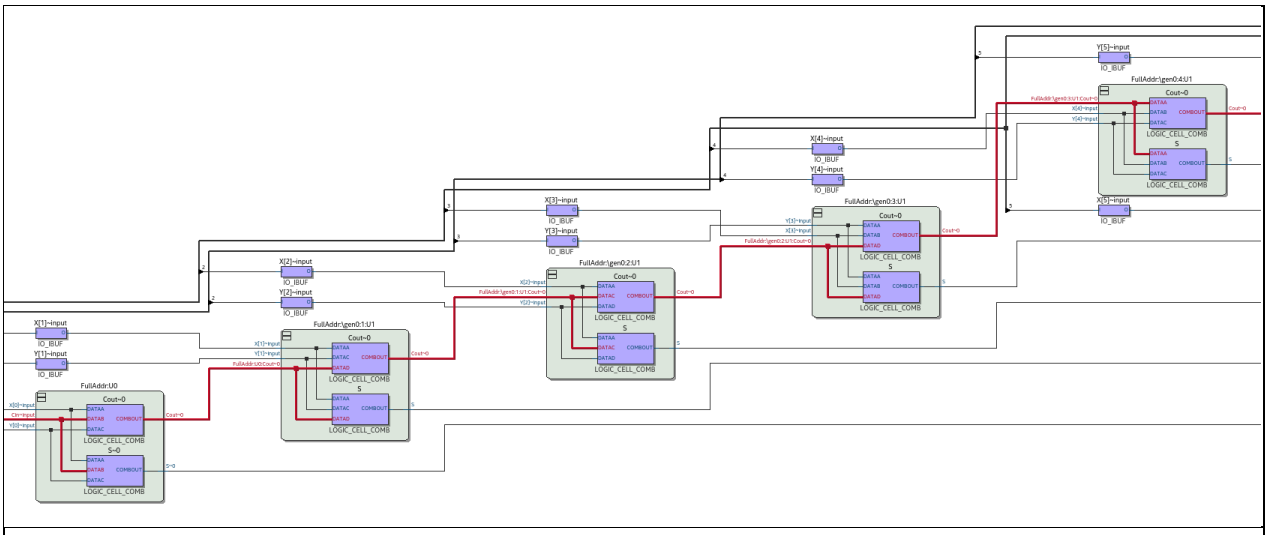

Figure 4.1.2: TVPF: First Five Stages of the Baseline Adder When $N = 21$

4.2 Design Candidate 1

Figure 4.2.1 shows a close-up technology post-fit view of the CSAN topology implemented on a Cyclone IV FPGA. Highlighted is the base case full adder mentioned in section 3.2, implemented using two LEs. Unlike the baseline device where Quartus creates a chain of entities to fit the design topology on the FPGA, the recursively defined nature of the CSAN topology results in a hierarchical, tree-like structure. Estimating LE usage for CSAN is less direct than RCAN due to synthesis optimizations and resource packing; however, the recursive expressions from Section 3.2 provide an upper-bound estimate. This is demonstrated below.

1. $3\left(3\left(3\left(3(3(3(1)))\right)\right)\right) = 729$ full adders.
2. $33 + 3\left(17 + 3\left(9 + 3\left(5 + 3(3 + 3(2))\right)\right)\right)$ =1029 2-channel, 1-bit multiplexers.

If each full adder entity uses 2 logic elements, and each multiplexer entity uses 1 logic element, theoretically, we would need a vast number of resources: 2487 logic elements, to be exact.
This gap indicates that Quartus is heavily optimizing and/or packing logic into available structures rather than mapping one-to-one with the abstract full-adder/mux count.

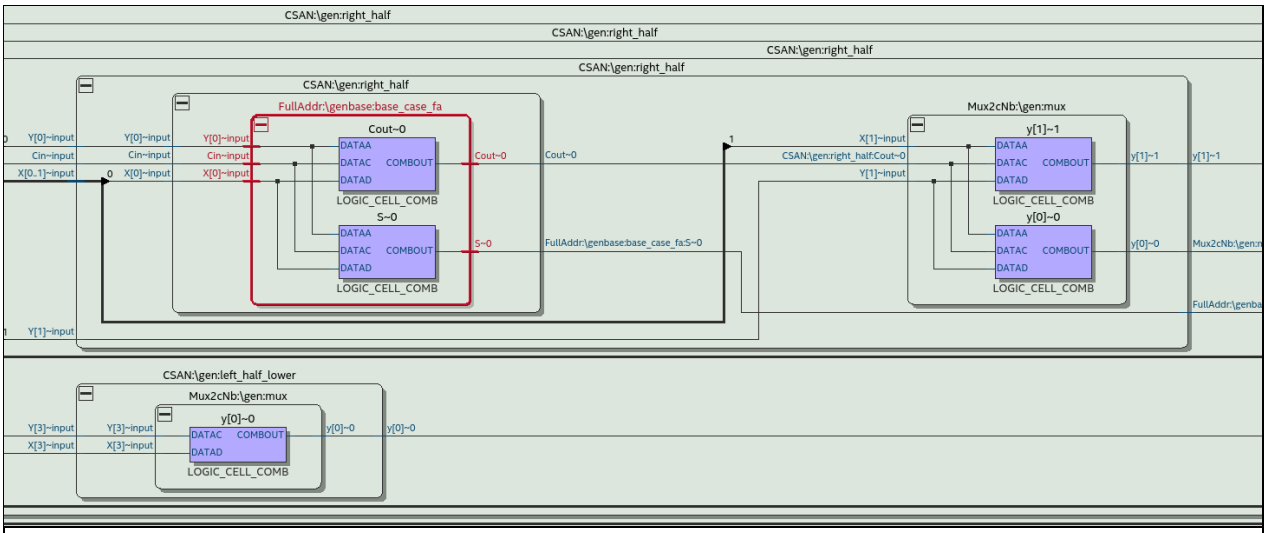

Figure 4.2.1: TVPF: Close-Up of Design Candidate 1

4.3 Design Candidate 2

Figure 4.3.1 shows the technology post-fit view of the first three stages of the RCAN topology implemented on an Arria II FPGA.

1. The first stage produces a sum while using 1 ALM.
2. The second stage produces a carry-out and sum while using 2 ALMs.
3. The third stage produces a sum while using 1 ALM.

Using the same logic as the baseline device, this design candidate should implement the RCAN topology on this FPGA using $N * 2$ ALUTs, or N ALMs. However, Quartus uses 133 ALUTs instead of the expected 128.

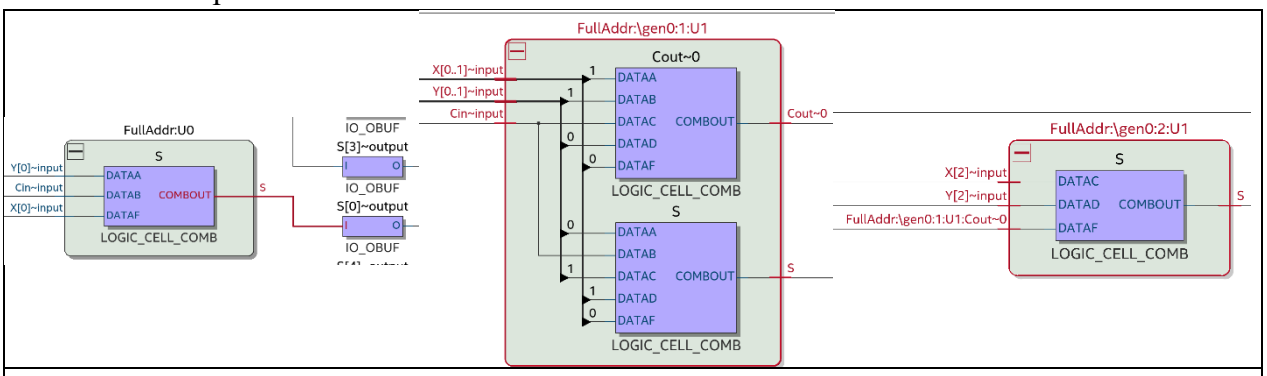

Figure 4.3.1: TVPF: Left to Right: First, Second, and Third Stages of Design Candidate 2

4.4 Design Candidate 3

Figure 4.4.1 shows a close-up technology post-fit view of the CSAN topology implemented on an Arria II FPGA. Due to the differences in ALMs and LEs, it is very different from the close-up of the topology implemented on the Cyclone IV FPGA. Using the same logic here as Design Candidate 1, the total number of ALUTs should also be 2487, however, Quartus only uses 209 ALUTs to fit the CSAN topology on this FPGA.

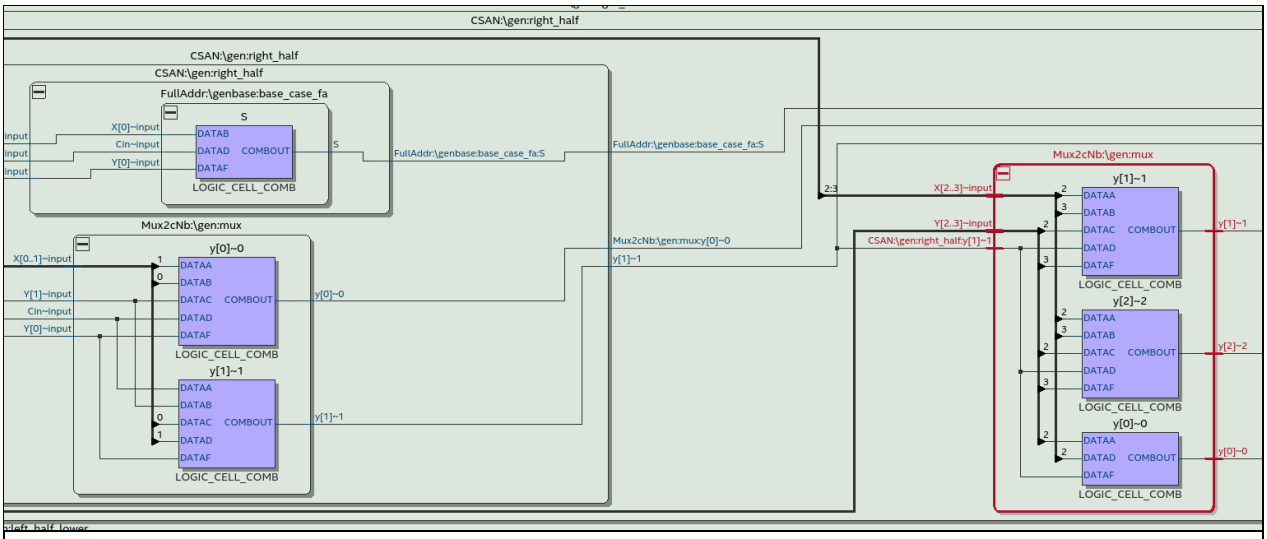

Figure 4.4.1: TVPF: Close-Up of Design Candidate 3

5.0 Results

The purpose of the following sections is to perform a cost-benefit analysis on all design candidates, provide details on the strengths and weaknesses of each one, and give a final statement which details the optimal use case for each design candidate.

5.1 Performance Comparison

An in-depth performance analysis for each design candidate is not within the scope of this report, and will be covered in the next stage of this project, however, at the moment, we may perform some basic estimates regarding the speed of the design candidates, with the following facts:

1. The Cyclone IV FPGA has a speed rating of 7. According to the Cyclone IV device handbook[4], this means that it has a maximum input clock frequency, or clock rate, of 472.5 MHz.
2. The Arria II FPGA has a speed rating of 6. According to the Arria II Device Handbook[5] Volume 3: Device Datasheet and Addendum, this means that it has a maximum input clock frequency of 500 MHz.

At this stage, we can estimate that a circuit fitted onto the Arria II FPGA is thus $\frac{500}{472.5} \approx 1.06$ times faster than the same circuit fitted onto the Cyclone IV FPGA.

Also, let us assume that the theoretical propagation delays derived in sections 3.1 and 3.2 can accurately represent the propagation delay of the circuit when fitted onto the FPGA[6].

Recall the theoretical propagation delay, P_d, for the RCAN and CSAN topologies:

1. $P_{dRCAN} = (2N + 1)\, D_{FA}$. For $N = 64$, $P_{dRCAN} = 129 D_{FA}$
2. $P_{dCSAN} = D_{FA} + D_M * \log_2 N$. For $N = 64$, $P_{dCSAN} = D_{FA} + 6D_M$

From these two expressions we can derive the expressions in Figure 5.1.1 and 5.1.2:

$$\frac{\mathrm{P}_{DC_1}}{\mathrm{P}_{DC_0}} = \frac{\mathrm{P}_{DC_3}}{\mathrm{P}_{DC_2}} = \frac{\mathrm{P}_{CSAN}}{\mathrm{P}_{RCAN}} = \frac{P_{dRCAN}}{P_{dCSAN}} = \frac{129 D_{FA}}{D_{FA} + 6D_M} = \frac{129}{1 + 6\left(\frac{D_M}{D_{FA}}\right)}$$

Figure 5.1.1: Theoretical Speedup of the CSAN Compared to the RCAN
D_{FA}: Delay of a Full Adder
D_M: Delay of a 2-Channel, 1-Bit Multiplexer
DC_x: Design Candidate {x}. If x = 0, This is the Baseline Device

$$\mathrm{P}_{DC_2} = 1.06\mathrm{P}_{DC_0}, \mathrm{P}_{DC_3} = 1.06\mathrm{P}_{DC_1}$$

Figure 5.1.2: Cross-FPGA Performance Metrics

[4]Cyclone IV Device Handbook Volume 3.
[5]Arria II Device Handbook Volume 3: Device Datasheet and Addendum.
[6] In reality, this is not even close to being accurate, due to the way that Quartus optimally fits the circuit, almost entirely destroying the original structure seen in the RTL view, from which we derived these propagation delay expressions.

Let us assume that $D_M \ll D_{FA}$. In this case, the speedup of the CSAN compared to the RCAN is ≈ 129, which is our maximum theoretical speedup. Conversely, if $D_M \gg D_{FA}$, the speedup would instead be ≈ 0. Assuming that a full adder is implemented with three levels of gates, and that a 2-channel, 1-bit multiplexer is implemented with two levels of gates, we can then say that $D_M =$ 2ns and $D_{FA} =$ 3ns. With the above expressions, we may then derive Table 5.1.2:

Table 5.1.2: Normalized Performance Metrics for Each Design Candidate

	Baseline Device	Design Candidate 1	Design Candidate 2	Design Candidate 3
Normalized Performance	1	25.8	1.06	27.348

5.2 Cost Comparison

To perform a cost comparison between the design candidates, we will first make the following initial estimates, in regards to cost:

1. $1\ ALUT = 1.33\ LE$s. In arithmetic mode, 1 ALUT has two 4-input LUTs, and 1 LE has two 3-input LUTs. Roughly, since $\frac{2*4}{2*3} \approx 1.33$, we can make this estimate.
2. $1\ ALM = 2\ ALUTs = 2.66\ LEs$. This follows from the statement made in the Arria II manual stating that 1 ALM consists of two combination ALUTs, and the above estimate.

To enforce the legitimacy of these initial estimates, let us first look at Table 5.2.1, containing the raw resource utilization metrics for each design candidate, for several values of N:

Table 5.2.1: Raw Resource Utilization Numbers for Each Design Candidate

	Resource Utilization: N=4	Resource Utilization: N=8	Resource Utilization: N=16	Resource Utilization: N=32	Resource Utilization: N=64	Resource Utilization: N=128
Baseline	9 LEs	17 LEs	33 LEs	73 LEs	155 LEs	311 LEs
Design Candidate 1	10 LEs	22 LEs	57 LEs	134 LEs	291 LEs	620 LEs
Design Candidate 2	7 ALUTs= 4 ALMs	15 ALUTs= 8 ALMs	31 ALUTs= 16 ALMs	67 ALUTs= 34 ALMs	133 ALUTs= 67 ALMs	274 ALUTs= 137 ALMs
Design Candidate 3	7 ALUTs= 4 ALMs	21 ALUTs= 11 ALMs	48 ALUTs= 24 ALMs	92 ALUTs= 46 ALMs	209 ALUTs= 105 ALMs	414 ALUTs= 212 ALMs

Assuming that Design Candidate 1, and Design Candidate 3 should use approximately the same number of resources since they use the same design topology, and that the Baseline Device should use approximately the same number of resources as Design Candidate 2 by the same logic, we may then construct Table 5.2.2:

Table 5.2.2: Ratio of LEs (Cyclone IV) to ALMs (Arria II) for the Two Topologies

	LE:ALM Ratio: N=4	LE:ALM Ratio: N=8	LE:ALM Ratio: N=16	LE:ALM Ratio: N=32	LE:ALM Ratio: N=64	LE:ALM Ratio: N=64	Average LE:ALM Ratio

RCAN	9 LEs : 4 ALMs = 2.2500	2.1250	2.0625	2.1471	2.3134	2.2701	2.1947
CSAN	10 LEs : 4 ALMs = 2.5000	2.0000	2.375	2.9130	2.7714	2.9245	2.5807

We may then derive an overall average LE:ALM ratio of 2.3877. This overall average is misleading because RCAN and CSAN map differently onto ALMs/LEs. Furthermore, the actual ratio is much less than our initial guess. The reasons for these discrepancies lie in the numerous optimizations performed by Quartus when fitting the circuit, as well as the fact that 4 4-input LUTs (1 ALM) is not directly equivalent to $\frac{16}{3}$ 3-input LUTs. Let us, for the sake of simplicity in this report, average our theoretical and experimental LE:ALM ratios, to acquire our actual LE:ALM ratio, $k = 2.5239$. Using this ratio, we can derive the expressions in Figures 5.2.1 and 5.2.2:

$$N_{LEs,Design\ Candidate\ x} = kN_{ALMs,Design\ Candidate\ x}$$

Figure 5.2.1: ALM to LE Conversion Using k for Design Candidates 2 and 3

$$Normalized\ Cost_{Design\ Candidate\ x,Baseline} = \frac{N_{LEs,Design\ Candidate\ x}}{N_{LEs,Baseline}}$$

Figure 5.2.2: Normalized Cost Formula

With the above expressions, we can then construct Table 5.2.3:

Table 5.2.3: Normalized Cost Metrics for Each Design Candidate

	Baseline Device	Design Candidate 1	Design Candidate 2	Design Candidate 3
Normalized Cost	1	1.8774	1.0910	1.7097

5.3 Concluding Statement

Table 5.3.1 displays the normalized cost and performance metrics for each design candidate.

Table 5.3.1: Cost-Performance, Performance-Cost Ratios for Each Design Candidate

	Normalized Cost	**Normalized Performance**	**Cost-Performance Ratio**	**Performance-Cost Ratio**
Baseline Device	**1**	**1**	**1**	**1**
Design Candidate 1	**1.8774**	**25.8**	**0.0728**	**13.7424**
Design Candidate 2	**1.0910**	**1.06**	**1.0292**	**0.9716**
Design Candidate 3	**1.7097**	**27.348**	**0.0625**	**15.9958**

Based on the normalized performance–cost ratio reported in Table 5.3.1, Design Candidate 3 (CSAN implemented on the Arria II FPGA) provides the best overall tradeoff for general-purpose and performance-driven applications. Conversely, for cost-constrained designs, the Baseline Device (RCAN implemented on the Cyclone IV FPGA) remains the preferred option due to its lowest resource cost.

6.0 Appendix

6.1 VHDL Listing - DP1-VHDL-Listing-G12-350-1261.pdf

The full PDF document containing all VHDL sourcecode, with coloured syntax highlighting and line numbers, is located in the ./Documentation directory, named:
"DP1-VHDL-Listing-G12-350-1261.pdf".
In it contains the complete vhdl source code for this project.